

Welcome to My Dev Talk 🚀

Powered by **Slidev** + **Catppuccin Mocha** 🐱

Code Highlighting – TypeScript

Step-by-step line highlighting with click:

```
1  // Define a generic Result type
2  type Result<T, E = Error> = { ok: true; value: T } | { ok: false; error: E }
3
4  function divide(a: number, b: number): Result<number, string> {
5      if (b === 0) return { ok: false, error: 'Division by zero' }
6      return { ok: true, value: a / b }
7  }
8
9  const result = divide(10, 3)
```

Code Highlighting – Python

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class Developer:
    name: str
    language: str
    coffee_level: int = 100

    def code(self) → Optional[str]:
        if self.coffee_level > 0:
            self.coffee_level -= 10
            return f"{self.name} writes beautiful {self.language}"
        return None
```

Left: Explanation

This layout splits content into **two columns**.

Perfect for:

-  Explaining code
-  Showing before/after
-  Comparing approaches

Right: Code

```
// Reactive state management
const state = reactive({
  count: 0,
  doubled: computed(() =>
    state.count * 2
  ),
})

function increment() {
  state.count++
}
```

Shell Commands

```
# Setup a new project  
pnpm create slidev@latest my-talk  
cd my-talk
```

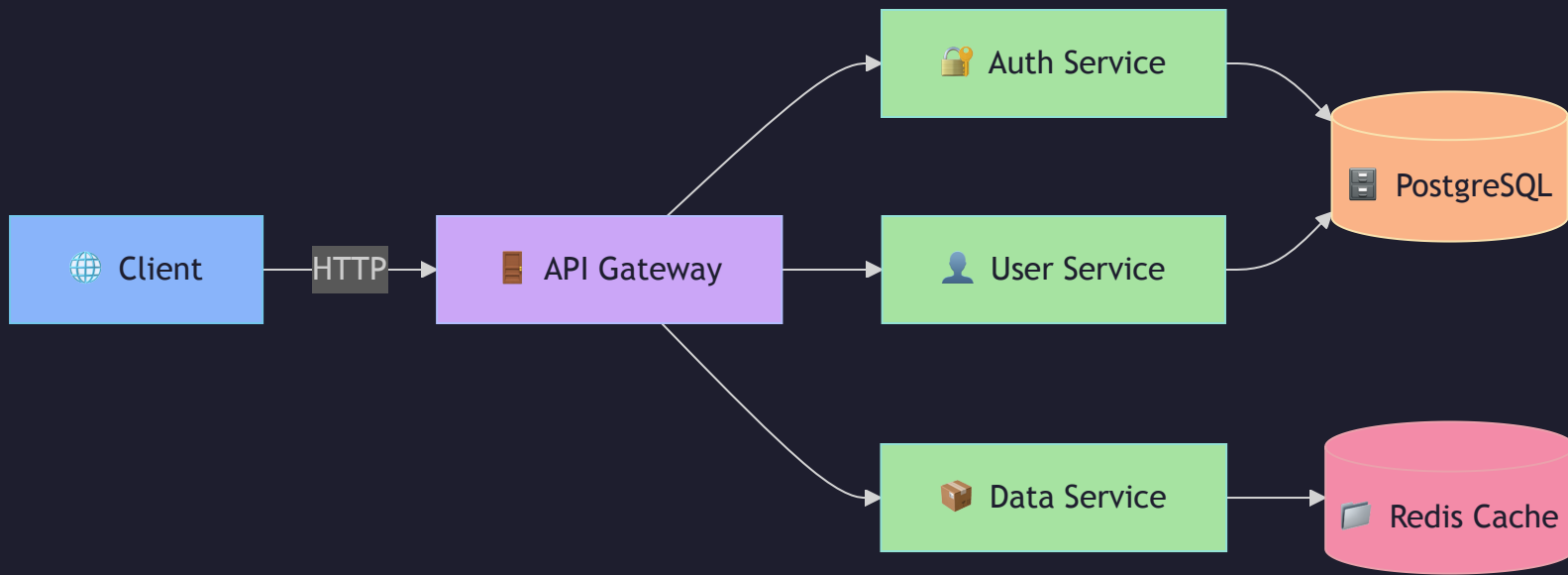
```
# Dev mode with hot reload  
pnpm dev
```

```
# Export to PDF  
pnpm slidev export
```

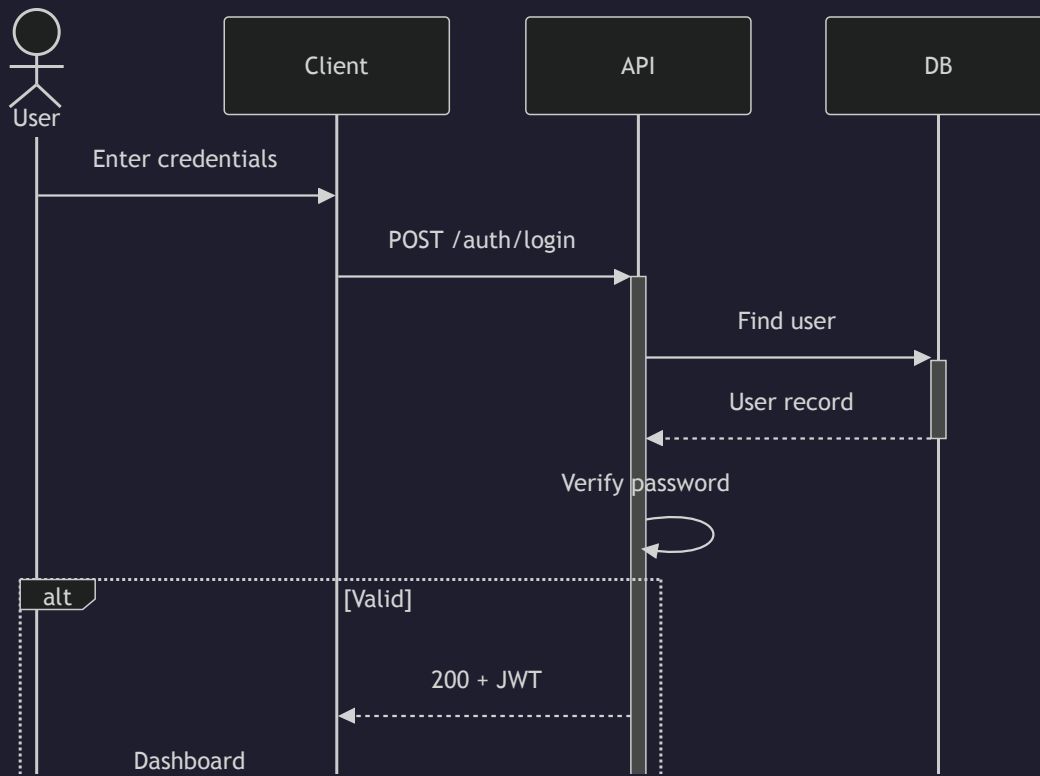
```
# Build for static hosting  
pnpm slidev build
```

Diagrams – Flowchart

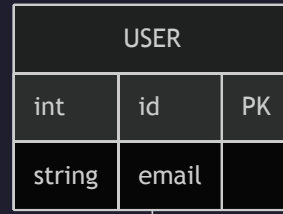
Architecture overview with Mermaid:



Diagrams – Sequence



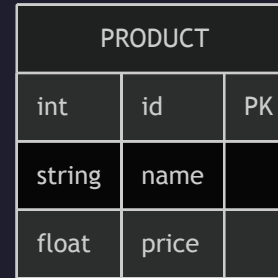
Diagrams – ER Diagram



places

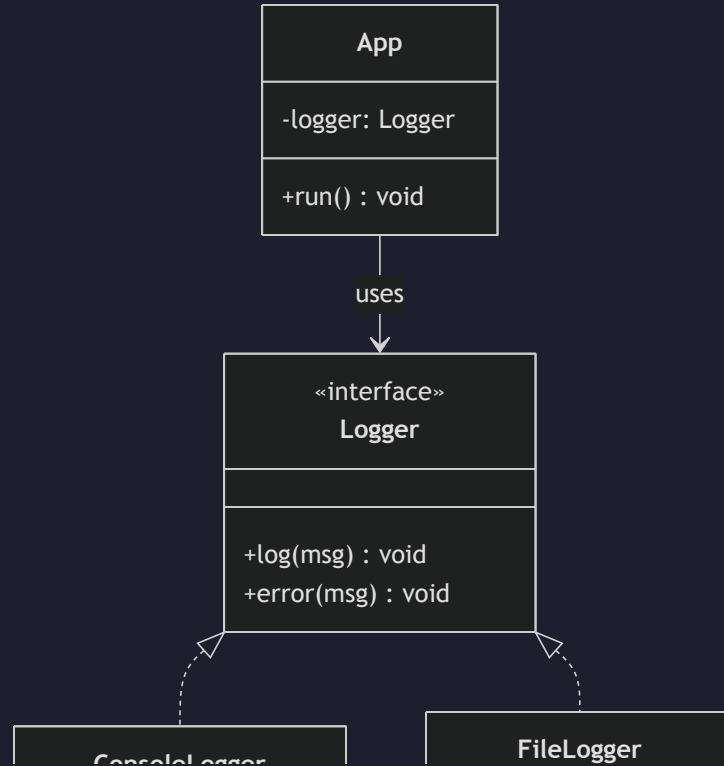


contains

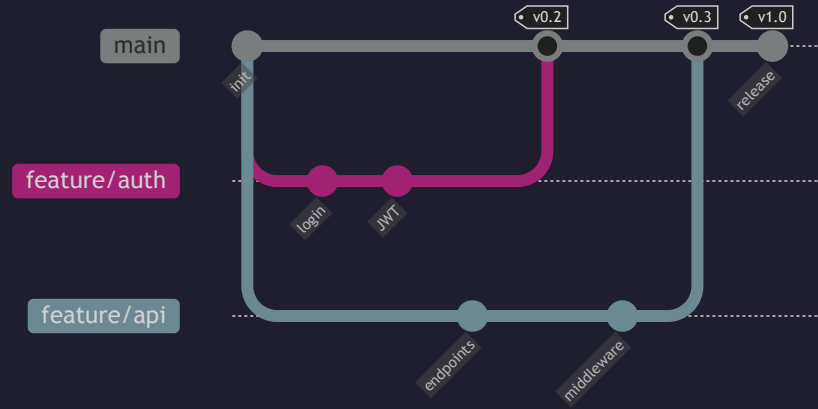


ordered in

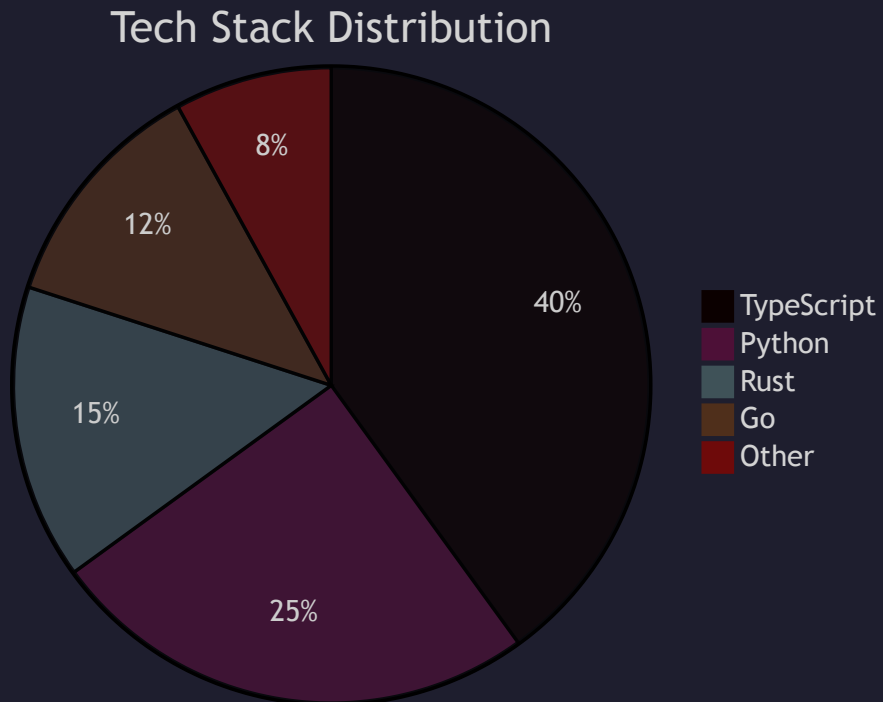
Diagrams – Class Diagram



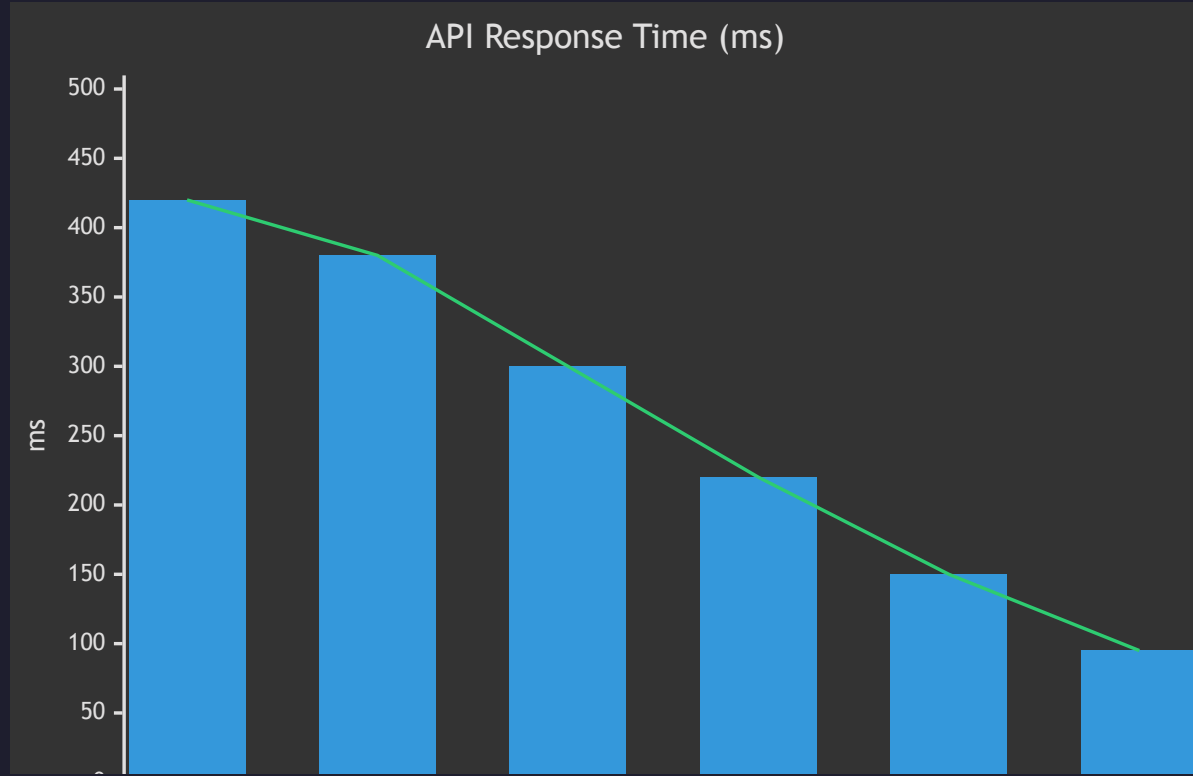
Diagrams – Git Graph



Charts – Pie

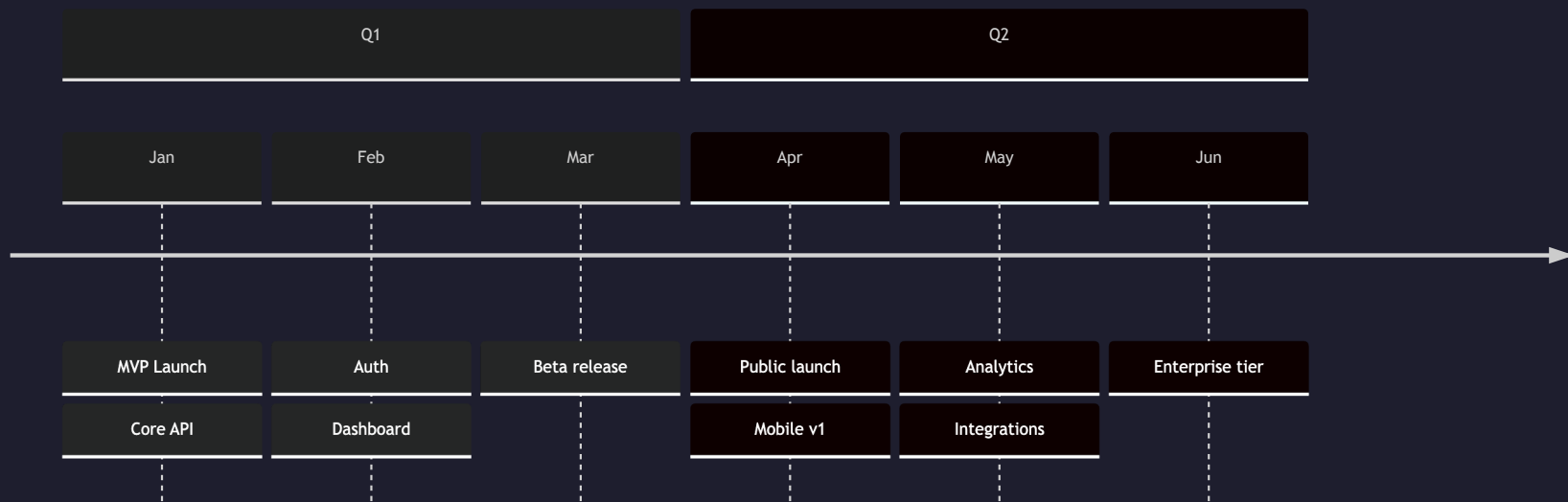


Charts – Bar & Line



Charts – Timeline

Product Roadmap 2026



Tables – Markdown

Simple and clean:

Feature	Status	Priority	Owner
Authentication	✅ Done	🔴 High	@alice
REST API	✅ Done	🔴 High	@bob
WebSocket	🔄 WIP	🟡 Medium	@charlie
Dashboard UI	🔄 WIP	🟡 Medium	@alice
Mobile App	❌ Todo	🟢 Low	@bob
CI/CD Pipeline	✅ Done	🔴 High	@charlie
Documentation	🔄 WIP	🟡 Medium	@alice

Tables – Styled HTML

Service	Status	Uptime	Latency
 API Gateway	 Healthy	99.98%	12ms
 Auth Service	 Healthy	99.95%	45ms
 User Service	 Degraded	98.50%	230ms
 Data Service	 Healthy	99.99%	8ms

Grid – Dashboard Cards

Total Users

12,847

↑ 12.5% vs last month

API Requests

1.2M

↑ 8.3% vs last month

Error Rate

0.12%

↓ 0.05% vs last month

Avg Response Time

45ms

↓ 15ms improvement

Deploy Frequency

4.2/day

↑ from 2.8/day

Grid – Tech Comparison



Bun

- All-in-one runtime
- Built-in bundler & test runner
- Fastest startup time
- Drop-in Node replacement

Best for: New projects, speed-critical



Deno

- Secure by default
- Native TypeScript
- Web-standard APIs
- Built-in formatter & linter

Best for: Security-focused, web standards



Node.js

- Largest ecosystem (npm)
- Battle-tested in production
- Massive community
- Enterprise adoption

Best for: Enterprise, existing projects



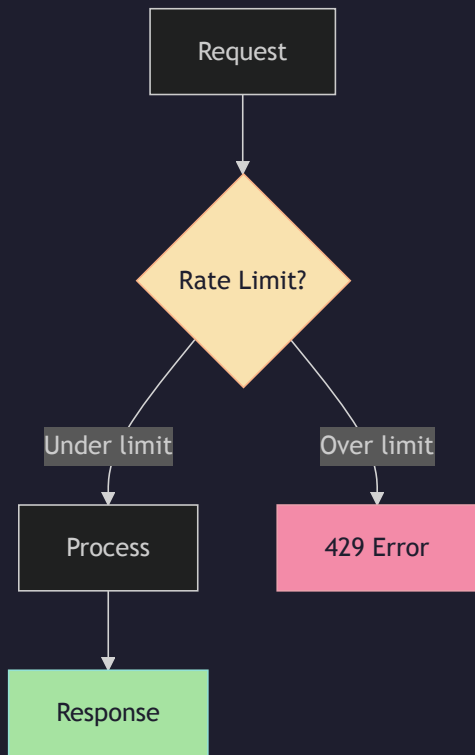
Cloudflare Workers

- Edge computing (0ms cold start)
- Global distribution
- V8 isolates
- Built-in KV & D1

Best for: Edge, low-latency APIs

Diagram + Code

Combine visuals with implementation:



```
const rateLimit = (
  max: number,
  windowMs: number
) => {
  const hits = new Map<string, number[]>()

  return (req: Request) => {
    const ip = req.headers
      .get('x-forwarded-for') ?? 'unknown'
    const now = Date.now()
    const window = hits.get(ip)
      ?.filter(t => t > now - windowMs)
      ?? []

    if (window.length ≥ max) {
      return new Response(
        'Too Many Requests',
        { status: 429 }
      )
    }

    hits.set(ip, [...window, now])
  }
}
```

Thank You! 🎉

Made with ❤️ using [Slidev](#)